

Cursors with SQL Server Part 1

By Don Schlichting

This series of articles will examine the purposes, uses, and optimization of cursors in SQL Server.

Introduction

SQL languages are designed so groups of records, or sets, can be manipulated easily and quickly. The speed at which groups of data can be altered, updated and deleted, demonstrates why working with sets is the preferred method. A traditional programmatic database connection, using ADO or ODBC, to manipulate large groups of records one at a time, will be magnitudes slower than executing a single SQL statement that will alter all the records as one giant set. But what happens when data must be organized, or adjusted on a record-by-record basis? Sometimes multiple processing steps are required on each individual record before moving on to the next. In these cases, the result set cannot be created by traditional SQL set statements. Enter Cursors. Cursors are special programming constructs that allow data to be manipulated on a row-by-row basis, similar to other structured programming languages. They are declared like a variable, and then move one record at a time using a loop for control. This article will examine several aspects of Cursor creation, use, and behavior.

Example One

In this first example, the task will be to alter the pubs titles database. Book prices need to be adjusted in such a way that any book with a price under \$20 be raised by 10%. While books currently \$20 or more, are raised 5%. If the following two simple set statements were used, incorrect results would occur.

```
UPDATE titles
SET price = price * (price * .1)
WHERE price < 20
```

```
UPDATE titles
SET price = price * (price * .05)
WHERE price >= 20
```

With these statements, any book with a price of \$19.95 will be raised 10%, then on the next statement, because the new price is over \$20, it will be raised an additional 5%. One solution would be to use a Cursor. (In the real world, a cursor would not be the best solution for this task, but it clearly demonstrates the topic. The [Optimization section](#) contains a preferred method for solving this example.) We will work through this example in several steps. To begin with, we will create a simple select to demonstrate the Cursor commands. Afterwards, we will continue to expand on it until the actual updates are completed. From Query Analyzer, enter the following TSQL:

```
USE pubs
GO
```

```
DECLARE get_price CURSOR FOR  
SELECT price FROM titles
```

```
OPEN get_price
```

```
FETCH NEXT FROM get_price
```

```
WHILE @@FETCH_STATUS = 0  
FETCH NEXT FROM get_price
```

```
CLOSE get_price
```

```
DEALLOCATE get_price
```

Executing this will produce a list of prices like the following:

	price
1	19.9900

	price
1	11.9500

	price
1	2.9900

	price
1	19.9900

Compare the above result with the result of just running the SELECT by itself, when it is not nested inside a Cursor.

```
SELECT price FROM titles
```

	price
1	19.9900
2	11.9500
3	2.9900
4	19.9900
5	19.9900
6	2.9900

The point worth noting is that in the first example, using the Cursor, multiple result sets are being returned. There are eighteen records in the titles table, so the Cursor will create eighteen independent result sets, each containing only one price. Running the select alone,

not inside the Cursor, returns one result set with eighteen records. This demonstrates that the Cursor is processing the table one record at a time, rather than working the table as one set.

DECLARE CURSOR FOR

The DECLARE Cursor statement is used for new cursor creation. Like other creating statements, we give the cursor a name, and then define how it will be used.

```
DECLARE get_price CURSOR FOR  
SELECT price FROM titles
```

The name of the Cursor, "get_price" in this example, can be any typical SQL identifier. In the actual select statement, only the field needing updating will be requested. The select can include all the usual TSQL commands such as WHERE, GROUP BY, and ORDER BY.

OPEN

```
OPEN get_price
```

The OPEN key word allocates memory for the new Cursor and does any other setup that might be required. While at the moment, it does not look like the OPEN statement is doing much, later, when Scrollable and Insensitive cursors are discussed, it is the OPEN statement that accomplishes all the housekeeping tasks required for them.

FETCH

```
FETCH NEXT FROM get_price
```

The purpose of FETCH is to actually retrieve a single record and load it into the cursor. In later examples, when Scrollable cursors are covered, we will have the option of replacing the NEXT keyword with direction pointers like PRIOR, FIRST, and LAST.

@@FETCH_STATUS

Cursors are usually controlled with a WHILE loop. The loop allows each record to be fetched one at a time. @@FETCH_STATUS is our loop control. There are three possible status values. 0 if the fetch was successful, -1 for an error, -2 for no records returned. So our loop will continue until the next fetch fails. The WHILE works like a traditional structured programming loop, fetching, checking the last status, repeat.

Cursors can also be nested, having one cursor call another. Alternatively, one cursor calls a stored procedure that opens another cursor. If this is the case, refer to Microsoft Books On Line keyword "@@FETCH_STATUS", because this function is global to the connection, not local to the loop. Care must be taken to insure you are checking the correct fetch status.

CLOSE

When records are fetched into a cursor, a cursor lock is issued. The lock is not released until a CLOSE is issued. After a CLOSE, the Cursor can be reopened. All the structure of the cursor is still in place.

DEALLOCATE

The last step in removing a Cursor is to release any memory its using, and destroy the variable and identifier name association. DEALLOCATE does all of this in one command. After this step, all of the cursor's structure is deleted.

Conclusion

With Cursors, TSQL statements can be created that mimic traditional structured programming, working on one record at a time, rather than working with sets. There will be a performance price to pay for this, but sometimes it may be required. In future articles, this cursor will be greatly expanded to include features like Scrolling (The ability to move forwards and backwards in a cursor), Insensitivity (using a copy of the select data, rather than the real data), and the use of Locking options, as well as discussions on cursor optimization.